Content Creator Guide — Derp Universe Game Definitions

This document is the single reference for everyone who authors or edits game content definitions. It explains every definition type, every field, every allowed value, and how things connect together. No server or API knowledge required.

> **Tip:** Use `Ctrl+F` / `⌘+F` to search this page. Every section starts with its admin table name in parentheses so you can search for it directly.

Table of Contents

Entities & Progression

- EntityDefinition — the species/character template
- EntityDefinition — `progression` — XP curves and stat growth
- NftCollectionDef — NFT-backed entity collection

Locations & Buildings

- LocationDefinition — outpost, region, or map-node template
- BuildingDef — building template for constructing inside locations

Missions

- MissionDef — Core Fields — mission template
- MissionDef — `tiers` — quality-tiered rewards (Gaussian rolls)
- MissionDef — Multi-Phase / Expedition Mode
- MissionDef — Modal Attachments
- MissionDef — `launchRequirements`

Crafting & Recipes

- RecipeDef — crafting recipe template

Items & Inventory

- ItemDefinition — Core Fields — item template
- ItemDefinition — `consumableData` — usable items
- ItemDefinition — `passiveEffects` — equipment passive bonuses

- ItemDefinition — `equipData` — equipment slot and stats
- ItemDefinition — `dustingData` — item breakdown / dusting
- LootboxDef — lootbox outcome pool

Rewards

- Outcome Entry Types — the universal reward format
- BundleDef — reusable reward bundle
- GiftDef — reusable gift template
- GiftSpec (Inline Gifts) — one-off gift on a definition

Currencies

- Currency — in-game currency definition

Store & Monetization

- ShopItemDef — individual store item
- ShopTabDef — store tab
- ShopSectionDef — section grouping within a tab
- ShopHeroDef — store hero banner carousel
- Gem Pack Display

Events, Telemetry & Daily Content

- EventDefinition — server-side event handler
- TelemetryDef — mission event counter
- CheckInDef — daily / weekly login reward track
- QuestDef and Achievements
- HomeBannerDef — home-screen rotating banner

Effects & Powers

- EffectDef — reusable effect template
- EffectCategory Enum
- EffectValueMode Enum
- EffectDurationModel Enum
- EffectStackingRule Enum
- EffectTargetType Enum
- EffectStatMap
- EffectDefOverrides
- PowerDef — global power template

UI & Modals

- ModalDef and DefaultModalDef — server-rendered modal
- DisplayDef — `style` — visual overrides on cards and banners

Synergy System

- SynergyRule — trait-combination bonus / penalty rule

Selection Rules

- SelectionRuleDef — reusable named condition
- RichSelection — Condition Tree — the universal condition format
- RichSelectionRule — Leaf Condition

Screens & Content Documents

- GameDefinition (Screen Compositions & Content Documents)
- Screen Composition Sections
- Content Document Blocks

System

- TagDef — content classification tag
- GameConstant — server-side tuning constant

Worked Examples

- Example 1 — Multi-Phase Expedition Mission with Tiered Rewards
- Example 2 — Consumable Item that Applies an XP Boost
- Example 3 — 7-Day Check-In Track with Inline Gifts

EntityDefinition

Admin table: `entityDefinition`

An EntityDefinition is the **blueprint for a character species**. Every entity that a player owns (whether minted from an NFT collection or created directly) is an instance of one of these definitions. The definition controls what the character looks like, its default stats, equipment slots, and how it levels up.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `derp_bird`. Never change after launch — it is stored on every entity instance.
`name`	string	✓	Display name shown to players.
`description`	string		Flavour text / lore shown in the entity detail panel.
`entityType`	string	✓	Broad type tag used by synergy selectors, e.g. `"character"` or `"creature"`.
`baseStats`	JSON object		Starting stats at level 1. Keys: `ATK`, `DEF`, `SPD`, `FOC`, `LCK`, `VIT`. Any stat left out defaults to `0`.
`progression`	JSON object		XP curve and stat gain per level. Omit to use the system defaults. See EntityDefinition — `progression`.
`equipmentSlots`	JSON array		Equipment slots this species can use, e.g. `["weapon", "armour", "accessory"]`. An empty array means no gear.
`displayDefId`	string		Links to a `DisplayDef` row for visual presentation.
`enabled`	boolean		`true` (default) = entities of this species can participate in all game systems. `false` = hidden from players.
`tags`	string[]		Classification tags for filtering in the admin panel.

Cross-references: - Equipment stat boosts: see ItemDefinition — `equipData` - NftCollectionDef links a policy ID to an `entityDefinitionId`: see NftCollectionDef

EntityDefinition — `progression`

The `progression` JSON blob on an EntityDefinition controls XP requirements and stat growth at every level. **Leave the whole field out to use the system defaults**, which give a sensible geometric XP curve and moderate stat scaling.

Top-level fields

Field	Type	Default	What it controls
`maxLevel`	number	`100`	The highest level entities of this species can reach.
`xpCurve`	object	`{ "kind": "geometric", "base": 100, "ratio": 1.15 }`	How XP requirements scale with level.
`statGain`	object	See sub-sections below	How stats grow at each level-up.

`xpCurve` — Geometric (default)

XP needed to advance from level N → N+1 is `floor(base × ratio^(N-1))`.

Field	Type	Default	What it controls
`kind`	`"geometric"`	required	Selects geometric scaling.
`base`	number	`100`	XP cost of the very first level-up (level 1 → 2).
`ratio`	number	`1.15`	How much more expensive each level is than the previous. `1.15` = +15% per level.

`xpCurve` — Table (hand-tuned)

Use this when you want exact control over each step. Entries beyond the array are capped at the last value.

Field	Type	What it controls
`kind`	`"table"`	Selects the manual table.
`perLevel`	number[]	XP cost of each level-up in order. Index 0 = level 1→2, index 1 = level 2→3, etc.

`statGain.guaranteed`

Fixed stat points every entity of this species gets on every level-up, regardless of rarity. Default is `+1` to all six stats.

```
{ "guaranteed": { "ATK": 1, "DEF": 1, "SPD": 1, "FOC": 1, "LCK": 1, "VIT": 1 } }
```

`statGain.bonusPool`

Extra points distributed on top of guaranteed. Two variants:

`rarityScaled` (default) — rarer entities earn bigger bonus pools at high levels.

Field	Default	What it controls
`kind`	`"rarityScaled"`	Selects this variant.
`base`	`14`	Bonus pool size for a common entity at level 1.
`exponent`	`0.16`	How much rarity amplifies the pool at higher levels. Higher = rarer entities scale further.

`flat` — same bonus pool for everyone regardless of rarity.

Field	What it controls
`kind`	`"flat"`
`amount`	Fixed bonus points per level-up for every entity.

`statGain.distribution`

Controls how the bonus pool is split between random draws.

Field	Default	What it controls
`uniformShare`	`0.5`	Fraction of the pool spread equally across all six stats.
`weightedShare`	`0.5`	Fraction distributed according to the entity's trait-defined weights. The two shares should add up to `1`.

NftCollectionDef

Admin table: NFT collections (managed via the NFT panel)

Links a blockchain NFT policy to a game definition so that holding the NFT gives you an entity, location, or item.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `derp_bird`.
`name`	string	✓	Display name for the collection.
`chain`	`"CARDANO"`	✓	Blockchain. Only Cardano is supported.
`collectionId`	string	✓	The Cardano policy ID.
`entityDefinitionId`	string	one of these three	Links to an `EntityDefinition`. Tokens mint as entity instances.
`locationDefinitionId`	string	one of these three	Links to a `LocationDefinition`. Tokens mint as location instances.
`itemDefinitionId`	string	one of these three	Links to an `ItemDefinition`. Tokens mint as items in inventory.
`mintMode`	`"one_per_token"`		How many instances are created per token. Only `one_per_token` is supported.
`data`	JSON object		Extra metadata (leave `{}` if unsure).

> Exactly one of `entityDefinitionId`, `locationDefinitionId`, or `itemDefinitionId` must be set — not more, not fewer.

LocationDefinition

Admin table: `locationDefinition`

A LocationDefinition is the blueprint for a **territory node** — an outpost, region hub, or map location. Players own instances of these locations and build structures inside them.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `outpost_forest`.
`name`	string	✓	Display name shown to players.
`description`	string		Flavour or lore text.
`locationType`	string		Classification, e.g. `"outpost"`, `"region"`. Used by synergy selectors.
`regionId`	string		Parent region ID this location belongs to.
`buildingSlots`	number		How many building slots the location has by default.
`displayDefId`	string		Links to a `DisplayDef` for visual presentation.
`enabled`	boolean		Default `true`.

BuildingDef

Admin table: `buildingDef`

A BuildingDef is the template for a **constructable structure** inside a location (forge, workshop, barracks, etc.).

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `forge`.
`name`	string	✓	Display name.
`description`	string		Flavour text.
`category`	string		Broad category, e.g. `"WORKSHOP"`, `"BARRACKS"`.
`maxLevel`	number		Highest upgrade level. Set via `rules.maxLevel`.
`buildCost`	`CostEntry[]`		Resources required to construct level 1. See Outcome Entry Types for cost format.
`buildDuration`	number		Seconds to build level 1.
`upgradeCost`	`CostEntry[]`		Resources per upgrade level.
`upgradeDuration`	number		Seconds per upgrade.
`tags`	string[]		Tags used by synergy selectors (`buildingTagsAny`).
`enabled`	boolean		Default `true`.

MissionDef — Core Fields

Admin table: `missionDef`

A MissionDef defines a **single mission** that players can send their entities on. Missions have a duration, a difficulty, optional costs, and produce rewards.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `mission_boss_alpha`.
`name`	string	✓	Display name.
`description`	string		Flavour text shown to players.
`missionType`	string		Type tag used by synergy contexts, e.g. `"quick"`, `"boss"`, `"seasonal"`.
`regionId`	string		The region the mission takes place in. Affects synergy context.
`difficulty`	number		Base difficulty score. Higher = harder success roll.
`minSuccessChance`	number		Minimum allowed success probability (0–100). Prevents teams from having zero chance.
`duration`	number		Base duration in seconds before synergy / effect modifiers.
`teamMin`	number		Minimum team size required to launch.
`teamMax`	number		Maximum allowed team size.
`cost`	`CostEntry[]`		Currencies or items consumed on launch. Omit for free missions.
`success`	`OutcomeBlock`		Rewards granted on success (flat outcome). Used when no `tiers` block is present.
`failure`	`OutcomeBlock`		Rewards granted on failure. Often empty or a consolation prize.
`tiers`	JSON object		Gaussian quality-tier rewards. Overrides the flat `success` block when present. See MissionDef — `tiers`.
`requirements`	`RichSelection`		Gates whether the mission row is **visible and clickable**. Checked at browse time. Omit to show to everyone.
`launchRequirements`	`RichSelection`		Gates whether the mission can actually be **started** given the selected team. Checked at launch time. See MissionDef — `launchRequirements`.
`tags`	string[]		Mission tags used by synergy rules, e.g. `["stealth", "boss"]`.
`mode`	`"single"` | `"multi_phase"`		`"single"` (default) = one-shot mission. `"multi_phase"` = expedition with multiple phases. See Multi-Phase.
`displayDefId`	string		Links to a `DisplayDef` for the mission category card visuals.

Field	Type	Required	What it controls
`categoryId`	string		Groups missions into a category.
`enabled`	boolean		Default `true`. Disabled missions are hidden from all players.
`sortOrder`	number		Display order within a category. Lower = shown first.

`leadInModalDefId`, `activeModalDefId`, `readyModalDefId`, `completedModalDefId`, `failedModalDefId`

These link to `ModalDef` rows that are shown at different stages of the mission lifecycle. See MissionDef — Modal Attachments.

MissionDef — `tiers`

When a mission defines a `tiers` object, **rewards are rolled using a Gaussian quality curve** based on the team's Mission Power instead of a flat success/failure outcome.

Tier keys

Each key in `tiers` corresponds to a quality level. Define only the tiers you want; missing tiers fall back to the flat `success.rewards` block.

Key	Gaussian center (default team MP)	Rarity feel
`failure`	—	Fires when the mission **fails**
`common`	15	Base quality
`uncommon`	45	Slightly better
`scare`	75	Mid-tier
`rare`	105	Good
`epic`	135	Excellent
`mythic`	165	Exceptional
`legendary`	195	Best possible

Each tier block shape:

```
{
  "rewards": [ /* OutcomeEntry array */ ],
  "clRequirement": 120
}
```

Field	What it controls
`rewards`	Reward entries for this tier. Same format as `success.rewards`.
`clRequirement`	Optional. Shifts the Gaussian peak for this tier to a different team MP value (default = the column above). Useful to make a tier reachable at lower or higher power.

Top-level `sigma`

Place `"sigma": <number>` at the root of the `tiers` object (alongside the tier keys) to control the probability spread:

- **Low sigma** (e.g. `10`) → outcomes cluster tightly around the best-matched tier. Good for predictable, skill-gated missions.
- **High sigma** (e.g. `40`) → outcomes spread across many adjacent tiers. Good for high-variance or lottery-feel missions.
- **Default:** `22` (moderate spread, adjacent tiers share ~21 % of the probability mass).

Probability at a glance (default sigma = 22)

Team MP	Common	Uncommon	Scare	Rare	Epic	Mythic	Legendary
15	69 %	28 %	2 %	—	—	—	—
75	2 %	21 %	54 %	21 %	2 %	—	—
195	—	—	—	—	2 %	28 %	69 %

MissionDef — Multi-Phase / Expedition Mode

Set `mode: "multi_phase"` on a MissionDef to create an expedition mission with sequential phases and decision points.

Mission-level fields (multi-phase only)

Field	Type	Default	What it controls
`phases`	`MissionPhaseDefinition[]`	—	Required. Ordered list of phase specs. Phase 0 overrides the top-level duration.
`rewardsMode`	enum	`"final_bundle"`	How rewards are granted.
`maxFailures`	number | null	`null`	Total phase failures before force-abandoning the whole mission. `null` = unlimited.

`rewardsMode` values

Value	What happens
`"final_bundle"`	One reward roll at the very end using the def-level `tiers`. Default.
`"per_phase"`	Each completed phase rolls its own `tiers`; no final roll.
`"hybrid"`	Per-phase rolls + a final bundle.

`MissionPhaseDefinition` shape

Each phase is a JSON object:

Field	Type	What it controls
`name`	string	Phase display name, e.g. `"Forest Approach"`.
`description`	string	Narrative flavour text for the phase decision modal.
`duration`	number	Seconds for this phase (overrides def-level duration for phase 0).
`difficulty`	number	Phase-level difficulty score.
`injuryRate`	number	Probability (0–1) of entities getting injured in this phase.
`injuryPenalty`	number	MP multiplier applied to injured entities (e.g. `0.8` = 80% effective). Entity VIT reduces both severity and duration.
`count`	number	How many times this phase repeats before moving on.
`tiers`	object	Per-phase reward tiers used when `rewardsMode` is `per_phase` or `hybrid`.
`continueRules`	object	What happens automatically after success / failure.

`continueRules` actions

Action	When	What happens
`"next_phase"`	on success or failure	Auto-advance to the next phase (or complete if it was the last).
`"complete"`	on success	Mark job complete; player collects rewards.
`"end_job"`	on failure	Abandon immediately; no final rewards.
`"wait"`	on success or failure	Pause at `phase_ready` — player must choose to continue or abandon.

MissionDef — Modal Attachments

Attach `ModalDef` rows to a mission to control what the player sees at each stage of the mission lifecycle.

Field	When it shows	Fallback
`leadInModalDefId`	Auto-opens once per session when the player views the mission. Player can always reopen via "Open Narrative" button.	None — if absent, no lead-in appears.
`activeModalDefId`	While the mission job is running.	System default.
`readyModalDefId`	When the mission is ready to collect.	Falls back to `activeModalDefId`.
`phaseDecisionModalDefId`	When a multi-phase job enters `phase_ready` (player decision point).	`DefaultModalDef` with id `default_mission_phase_ready`.
`completedModalDefId`	After success and reward collection.	System default.
`failedModalDefId`	After failure.	Falls back to `completedModalDefId`.

MissionDef — `launchRequirements`

`launchRequirements` is a `RichSelection` condition tree that is checked **at the moment the player taps "Launch"**, with the actual selected team in context. Use this when requirements depend on team composition (size, traits, stats).

Field	Checked when	Used for
`requirements`	Browse time (no team)	Level gates, flag checks, season gates. Drives the lock icon on the mission card.
`launchRequirements`	Launch time (real selected team)	Team size, entity trait requirements, stat thresholds. Both must pass to launch.

The top-level `label` on each rule node in `launchRequirements` is surfaced in the UI as a requirement status indicator.

RecipeDef

Admin table: `recipeDef`

A RecipeDef defines a **crafting recipe** — what inputs an entity consumes, how long it takes, and what the output is.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `recipe_iron_sword`.
`name`	string	✓	Display name.
`description`	string		Flavour text.
`data.tier`	number		Recipe tier (1–3). Determines which workshop level can run this recipe.
`cost`	`CostEntry[]`		Materials and currencies consumed per craft.
`duration`	number		Base duration in seconds.
`difficulty`	number		Pass/fail threshold. Higher = harder for low-luck entities.
`tiers`	JSON object		Quality-tier reward pool using the same Gaussian mechanic as missions. Entity crafting luck (CL) drives the quality roll. Same tier key names as missions (`common` → `legendary`).
`requirements`	`RichSelection`		Gates who can see and attempt this recipe.
`tags`	string[]		Classification tags.
`enabled`	boolean		Default `true`.

Cross-reference: Quality probabilities use the same tier/sigma/clRequirement system as MissionDef — `tiers`. The main difference is that the CL axis is the **individual entity's crafting luck** rather than team mission power.

ItemDefinition — Core Fields

Admin table: `itemDefinition`

An ItemDefinition is the **blueprint for any in-game item** — materials, consumables, equipment, lootboxes, or gift packages.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `mat_iron_ore`.
`name`	string	✓	Display name.
`description`	string		Flavour text.
`type`	enum	✓	Item category. See values below.
`rarity`	string		Rarity label, e.g. `"common"`, `"rare"`, `"legendary"`. Used for UI theming.
`maxStack`	number		Maximum quantity per inventory slot. `null` = unlimited.
`iconKey`	string		Asset key for the item icon.
`enabled`	boolean		Default `true`.
`lootboxDefId`	string		**Lootbox items only.** ID of the `LootboxDef` whose outcome pool is rolled when the player opens this item.
`consumableData`	JSON		**Consumable items only.** Effects applied when used. See `consumableData`.
`passiveEffects`	JSON		**Equipment items only.** Passive bonuses while equipped. See `passiveEffects`.
`equipData`	JSON		**Equipment items only.** Slot and stat bonuses. See `equipData`.
`dustingData`	JSON		Items with this set can be broken down. See `dustingData`.

`type` values

Value	What this item is
`MATERIAL`	Crafting ingredient or collectible resource. No special behaviour.
`CONSUMABLE`	Can be used from inventory to trigger effects. Requires `consumableData`.
`EQUIPMENT`	Can be equipped to entities. Requires `equipData` and optionally `passiveEffects`.
`LOOTBOX`	Opens to reveal randomised rewards. Requires `lootboxDefId`.
`GIFT_PACKAGE`	A wrapped gift that must be opened. Created when a `GiftDef` has `instant: false`.
`KEY_ITEM`	Quest or progression item. No special behaviour beyond inventory tracking.

ItemDefinition — `consumableData`

Present on `CONSUMABLE` items. Defines what happens when the player uses the item.

```
{
  "consumableData": {
    "effects": [
      {
        "effectDefId": "xp_boost_30min",
        "targetType": "player",
        "effectOverrides": { "value": 50, "durationSec": 1800 }
      }
    ]
  }
}
```

Field	Required	What it controls
`effects`	✓	Array of `ConsumableEffectSpec` entries applied on use.
`effects[].effectDefId`	✓	Which `EffectDef` to activate.
`effects[].targetType`	✓	Who receives the effect. See EffectTargetType Enum.
`effects[].effectOverrides`		Override specific fields of the `EffectDef` for this item only. See EffectDefOverrides.

ItemDefinition — `passiveEffects`

Present on `EQUIPMENT` items. Defines **bonuses that apply while the item is equipped** to an entity. These do not require a separate `EffectDef` row and do not create `ActiveEffect` records — they are resolved on the fly.

```
{
  "passiveEffects": [
    { "category": "stat_boost", "valueMode": "flat_add", "stats": { "ATK": 10, "DEF": 5 } },
    { "category": "xp_multiplier", "valueMode": "percent_add", "value": 15 }
  ]
}
```

Field	Required	What it controls
`category`	✓	Which game system is affected. See EffectCategory Enum.
`valueMode`	✓	How the value is applied. See EffectValueMode Enum.
`value`		Numeric magnitude. Required for most categories. Omit for `stat_boost` (use `stats` instead).
`stats`		Per-stat distribution for `stat_boost`. See EffectStatMap.
`trait`		Trait archetype to grant. Use for `trait_grant` category.

ItemDefinition — `equipData`

Present on `EQUIPMENT` items. Specifies which **equipment slot** the item occupies and its direct stat bonuses.

```
{
  "equipData": {
    "slot": "weapon",
    "stats": { "ATK": 15, "SPD": 3 }
  }
}
```

Field	What it controls
`slot`	Equipment slot identifier. Must match a slot defined on the target entity's species (e.g. `"weapon"`, `"armour"`, `"accessory"`).
`stats`	Map of stat → bonus value added while equipped. Stacks with `passiveEffects`.

> Only one item can occupy a given slot at a time. Equipping a second item in the same slot unequips the first.

ItemDefinition — `dustingData`

Present when the item can be **broken down** (dusted) by the player in exchange for resources. The dusting job uses the same quality-tier mechanic as crafting.

Field	Type	What it controls
`instant`	boolean	`true` = rewards granted instantly with no timer. `false` = creates a timed job the player must collect.
`timer`	Timer object	Duration and skip settings. Ignored when `instant: true`.
`timer.duration`	number	Duration in seconds.
`timer.skipable`	boolean	Whether the player can skip the timer with currency.
`timer.skip_cost`	array	`[{ type, id, amount }]` cost to skip.
`timerScaling`	`"flat"` | `"per_unit"`	`"flat"` = fixed duration regardless of quantity. `"per_unit"` = `duration × quantity`.
`difficulty`	number	Pass/fail threshold. `0` = always succeed.
`tiers`	object	Quality-tier rewards: `failure`, `common`, `uncommon`, `scare`, `rare`, `epic`, `mythic`, `legendary`. Same format as mission tiers.

LootboxDef

Admin table: `lootboxDef`

A LootboxDef defines the **outcome pool** that is rolled when a player opens a lootbox item. The `ItemDefinition` with `type: LOOTBOX` links to this def via `lootboxDefId`.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `lootbox_standard_pool`.
`name`	string	✓	Admin-facing display name.
`outcomes`	`OutcomeEntry[]`	✓	The reward pool. Same format as mission rewards. Supports bundles, pick-mode, and `nothing` entries.
`enabled`	boolean		Default `true`.

Cross-reference: Outcome format is described fully in Outcome Entry Types.

Outcome Entry Types

Used in: mission `success` / `failure`, mission `tiers`, recipe `tiers`, lootbox `outcomes`, bundle `bundle`, gift `rewardSpec`, quest `rewardSpec`, dusting `tiers`.

Every reward list is an array of outcome entries. Each entry has a `type` field plus type-specific fields.

Common field: `odds`

Mode	What `odds` means
Flat resolve	A percentage gate (0–100). The entry fires if a random roll ≤ `odds`. Default 100 = always fires.
Pick-mode bundle	A relative weight used by a weighted-random picker. Not a percentage.

Entry types

`type`	Required fields	What it grants
`currency`	`key` (currency ID), `min`, `max`	Grants `min`–`max` units of the currency.
`item`	`key` (item definition ID), `min`, `max`	Grants `min`–`max` copies of the item.
`entity_xp`	`min`, `max`	Grants XP split across all mission entities.
`keeper_xp`	`min`, `max`	Grants XP to the player account (Keeper level).
`flag`	`operation`, `key`, `value`	Mutates a **persistent** player flag. Survives across sessions.
`tempflag`	`operation`, `key`, `value`	Mutates an **ephemeral** flag scoped to the current resolution only. Use `scope: "tempflag"` in requirements to read it within the same resolution.
`text`	`message`	Picks a weighted flavour-text string for the result modal.
`emitEvent`	`defId`, `payload`, `idempotencyKey`	Dispatches a named game event (e.g. to issue a gift).
`nothing`	—	No-op weight anchor. Use in pick-mode bundles to represent "no reward".
`bundle_ref`	`bundleDefId`	Expands a `BundleDef` row inline.

`flag` and `tempflag` operations

Operation	What it does
`set`	Sets the flag to `value`.
`unset`	Removes the flag key.
`increment`	Adds `value` to the flag's current number.
`decrement`	Subtracts `value` from the flag's current number.
`invert`	Flips a boolean flag.
`remove`	Removes the flag key (same as `unset`).

Inline bundle wrapper

Wrap entries in a `bundle` array to create a local bundle. Add `pickMin` / `pickMax` for pick-mode:

```
{
  "bundle": [
    { "type": "item", "key": "mat_gold_core", "min": 1, "max": 1, "odds": 5 },
    { "type": "nothing", "odds": 95 }
  ],
  "pickMin": 1,
  "pickMax": 1
}
```

In this example, the player has a 5 % chance of getting the item and a 95 % chance of nothing.

BundleDef

Admin table: `bundleDef`

A BundleDef is a **reusable reward bundle** referenced from multiple outcome arrays via `{ "type": "bundle_ref", "bundleDefId": "..." }`. Use a BundleDef when the same set of possible rewards appears in several places (e.g. a "standard loot drop" shared by many missions).

Field	Type	Required	What it controls
`id`	string	✓	Unique slug.
`name`	string	✓	Admin-facing display name.
`bundle`	`OutcomeEntry[]`	✓	The reward entries in the bundle. Same format as any other outcome array.
`pickMin` / `pickMax`	number		If set, the bundle is resolved in pick-mode — draws `pickMin`–`pickMax` entries by weighted selection.
`enabled`	boolean		Default `true`. Disabled bundles produce no rewards when referenced.

GiftDef

Admin table: `giftDef`

A GiftDef is a **reusable gift template** stored in the database. Reference it anywhere you need the same gift from multiple places (events, check-in tracks, admin tooling) using `{ "giftDefId": "..." }`.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `gift_welcome_pack`.
`name`	string	✓	Admin-facing display name.
`version`	integer		Monotonically increasing version. The system manages this.
`presentation`	object		Visual hints for the gift card and reveal. See sub-fields below.
`claimRules`	object		Constraints on issue and claim. See sub-fields below.
`rewardSpec`	`OutcomeEntry[]`		Rewards granted on claim. Same format as mission rewards.
`instant`	boolean		`true` (default) = rewards granted immediately. `false` = a `GIFT_PACKAGE` item is added to inventory for later opening.
`modalDefId`	string		Optional `ModalDef.id` shown when the player opens the gift.
`enabled`	boolean		Default `true`.

`presentation` sub-fields

Field	What it controls
`title`	Main heading on the gift card and notification.
`subtitle`	Optional secondary text below the title.
`iconKey`	Asset key for the gift icon.
`animationKey`	Asset key for the reveal animation played on claim.
`accentColor`	Hex colour for the card accent, e.g. `"#FFD700"`.

`claimRules` sub-fields

Field	What it controls
`limitPerPlayer`	Maximum times a single player may claim this gift. Omit for unlimited.
`expiresInSeconds`	Seconds from issue time until the gift expires. Expired gifts cannot be claimed. Omit for no expiry.

GiftSpec (Inline Gifts)

Anywhere the system accepts a `GiftSpec`, you can either **reference an existing GiftDef** or **define a one-off gift inline** without creating a separate database row.

```
// Reference variant
{ "giftDefId": "gift_welcome_pack" }

// Inline variant (name key present, no giftDefId key)
{
  "name": "Day 1 Bonus",
  "presentation": { "title": "Welcome Back!", "iconKey": "gift_star" },
  "rewardSpec": [{ "type": "currency", "key": "soft_coins", "min": 100, "max": 100 }],
  "instant": true
}
```

Use a `GiftDef` row when the same gift is issued from multiple places.
Use an inline gift when the gift is unique to one definition and won't be reused.

`GiftSpec` **is accepted in:** - `CheckInDef.rewards[].gift` - `EventDefinition` handler payload for `createGiftForPlayer`

Currency

Admin table: `currency`

Currency rows define the **in-game currencies** players earn and spend.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `soft_arcanium`. Referenced everywhere currencies are used.
`name`	string	✓	Display name.
`symbol`	string		Short symbol or emoji shown next to the balance.
`iconKey`	string		Asset key for the currency icon.
`type`	enum	✓	Currency class. See values below.
`seasonId`	string		Season this currency belongs to (nullable).
`isTradeable`	boolean		Default `false`.
`isSpendable`	boolean		Default `true`. When `false`, `spend` calls are rejected — useful for display-only or accumulation currencies.
`decimals`	integer		Decimal precision shown in the UI. Default `0`.
`cap`	bigint		Maximum supply cap. `null` = uncapped.
`enabled`	boolean		Default `true`. Disabled currencies are excluded from all active game systems (balances remain).
`showInTopBar`	boolean		Default `false`. When `true`, shows this currency in the player's top bar. The bar shows at most **6** currencies.
`sortOrder`	integer		Position in the top bar. Lower = further left. Ties are broken alphabetically by `id`.

`type` values

Value	Description
`SOFT`	Earnable through regular gameplay (missions, production).
`HARD`	Slightly scarcer; obtained from specific activities.
`PREMIUM`	Real-money equivalent. Cannot be earned without purchase.
`SEASONAL`	Exists only during a specific season.
`EVENT`	Exists only during a limited-time event.
`GUILD`	Guild-level shared currency.
`CRYPTO`	Blockchain-backed token (CNT on Cardano).

ShopItemDef

Admin table: `shopItemDef`

A ShopItemDef is a **single purchasable item** in the player store.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `bundle_starter_pack`.
`name`	string	✓	Display name on the store card.
`description`	string		Short description.
`tab`	string (FK)		Which store tab this item appears in (`ShopTabDef.id`). Default `featured`.
`sectionId`	string (FK)		Optional section within the tab (`ShopSectionDef.id`).
`cost`	`CostEntry[]`		What the player pays. Array of `{ type, id, amount }`.
`originalCost`	`CostEntry[]`		Original price for strikethrough display (same format). Leave empty if not on sale.
`contents`	`ContentEntry[]`		What the player receives. See content entry types below.
`badge`	string		Small badge label on the card, e.g. `"HOT"`, `"NEW"`, `"SALE"`.
`badgeColor`	string		CSS background colour for the badge.
`imageUrl`	string		URL or asset key for the card image.
`displayOrder`	number		Sort order within the tab or section. Lower = first.
`enabled`	boolean		Default `true`. Hidden from all players when `false`.
`availableFrom`	datetime		ISO datetime — item is suppressed until this time. Leave `null` for immediate availability.
`limitedUntil`	datetime		ISO datetime — item is automatically removed after this time. A live countdown appears when fewer than 24 h remain.

`contents` entry types

`type`	`id`	`amount`	What is granted
`currency`	currency ID	number	Credits the player's balance.
`item`	item definition ID	number	Adds items to inventory.
`outpost_slots`	omit	number	Permanently unlocks additional outpost territory slots.

ShopTabDef

Admin table: `shopTabDef`

ShopTabDef rows define the **tabs** in the player store. Five canonical tabs are seeded automatically; you can add custom tabs.

Field	Type	Required	What it controls
`id`	string	✓	Unique tab identifier, e.g. `featured`, `gems`, `seasonal`. Used as the `tab` key on `ShopItemDef` rows.
`name`	string	✓	Label shown in the player-facing tab strip. Supports emoji, e.g. `"💎 Gems"`.
`tabType`	enum		`items` (default) — renders `ShopItemDef` rows. `gems_packs` — renders the gem pack + Derp Pass UI.
`layout`	enum		Card layout within the tab: `featured` (full-width), `grid-3` (3-column), `grid-2` (2-column, default).
`ungroupedPlacement`	enum		Where items without a `sectionId` appear: `before` (above sections, default) or `after`.
`sortOrder`	number		Left-to-right tab order. Lower = leftmost.
`enabled`	boolean		Default `true`.

ShopSectionDef

Admin table: `shopSectionDef`

Sections insert **named group headings** between items within a tab.

Field	Type	Required	What it controls
`id`	string	✓	Unique section identifier.
`tabId`	string (FK)	✓	The `ShopTabDef.id` this section belongs to.
`label`	string	✓	Section heading shown above the item group.
`subtitle`	string		Optional subtitle below the heading.
`sortOrder`	number		Vertical ordering within the tab. Lower = higher on screen.

ShopHeroDef

Admin table: `shopHeroDef`

Hero banners are a swipeable carousel at the very top of the store, above the tab bar.

Field	Type	Required	What it controls
`headline`	string	✓	Main banner text. Displayed in uppercase with the Cinzel font.
`subText`	string		Secondary line below the headline.
`imageUrl`	string		Background image URL or asset key (displayed at 160 px height).
`backgroundColor`	string		CSS background colour used when no image is set, or as overlay base. Default `#1a1640`.
`ctaLabel`	string		Call-to-action button label, e.g. `"Shop Now"`. Leave blank to hide the button.
`ctaTarget`	string		CTA destination: a store tab ID (e.g. `featured`) to switch tabs in-app, or a full `https://` URL to open externally.
`sortOrder`	number		Position in the carousel. Lower = appears first. Default `0`.
`enabled`	boolean		Disabled banners are hidden from all players. Default `true`.
`limitedUntil`	datetime		ISO datetime — banner automatically hides after this time.

Carousel behaviour: - 0 active banners → carousel is hidden; store opens at the tab bar. - 1 active banner → static card (no swipe, no dots). - 2–5 active banners → swipeable with dot indicators and 5-second auto-advance. Auto-advance pauses for 10 s after user interaction.

Gem Pack Display

Admin table: gem pack display metadata (managed via the Gem Packs admin panel)

Visual presentation fields for each gem pack (prices are set in Stripe).

Field	Type	What it controls
`displayName`	string	Label on the pack card, e.g. `"HANDFUL"`, `"SATCHEL"`, `"CHEST"`.
`accentColor`	string	CSS colour for the display name text.
`borderColor`	string	CSS colour / rgba for the card border.
`glowColor`	string	CSS colour for the card glow shadow. `"transparent"` = no glow.
`badgeType`	`"best_value"` | `"extra"` | `null`	Badge style: `null` = none, `"best_value"` = purple ribbon, `"extra"` = green custom label.
`badgeLabel`	string | null	Text inside an `"extra"` badge, e.g. `"10% EXTRA"`. Only shown when `badgeType === "extra"`.
`iconSize`	number	Gem icon size in pixels (32–50).

EventDefinition

Admin table: `eventDefinition`

An EventDefinition describes a **named server-side event** that can be dispatched to trigger backend logic on behalf of a player (e.g. issuing a gift when a mission completes).

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `event_mission_legendary_reward`.
`name`	string	✓	Admin-facing display name.
`kind`	string		Categorisation label for filtering in the admin panel. Informational only — has no effect on dispatch.
`handlerKey`	string	✓	Which server-side handler runs when this event is dispatched. See values below.
`security`	object		Access control. See sub-fields below.
`inputSchema`	JSON		Optional JSON Schema documenting the expected payload shape — for documentation and editor validation only.
`enabled`	boolean		Default `true`. Disabled events cannot be dispatched.

`handlerKey` values

Value	What it does	Required payload
`createGiftForPlayer`	Issues a gift to the target player.	`gift` — a `GiftSpec` (either `{ "giftDefId": "..." }` or a full inline gift).

`security` sub-fields

Field	What it controls
`allowedEmitters`	Array of `sourceType` strings that may dispatch this event. Common values: `"MissionJob"`, `"CraftingJob"`, `"BuildingJob"`, `"admin"`. Omit to allow any source.

TelemetryDef

Admin table: `telemetryDef`

A TelemetryDef defines a **real-time event counter** — useful for live event trackers, seasonal scoreboards, and community milestones. Each def can track several distinct sub-events simultaneously.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug (also used as the public URL key).
`name`	string	✓	Admin-facing display name.
`description`	string		Admin notes.
`enabled`	boolean		Default `true`.
`startDate`	ISO datetime		Events before this date are not counted. Omit to count from the beginning.
`endDate`	ISO datetime		Events after this date are not counted. Omit for no end.
`subEvents`	array		Counter definitions. See sub-event fields below.

Sub-event fields

Each sub-event is one atomic counter within the def.

Field	Type	Required	What it controls
`key`	string	✓	Unique key within this def (e.g. `"all_starts"`, `"wins"`).
`eventType`	enum	✓	Which game event increments this counter.
`filter`	`RichSelection` | `null`		Optional condition tree. Only events that pass the filter increment the counter. `null` = count all.

`eventType` values

Value	Triggered by
`MISSION_START`	A mission job being launched.
`MISSION_SUCCESS`	A mission completing with success.
`MISSION_FAIL`	A mission completing with failure.

CheckInDef

Admin table: `checkInDef`

A CheckInDef defines a **daily or weekly login reward track**. Players participate in all enabled tracks simultaneously and earn rewards on each claim.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `daily_login_track`.
`name`	string	✓	Admin-facing display name.
`description`	string		Admin notes.
`enabled`	boolean		Default `true`. Disabled tracks are ignored for all players.
`frequency`	object		Cadence object controlling how often the claim window resets. Default `{ "type": "daily" }`.
`cycleLength`	number		Number of reward slots in one cycle. After this many claims the streak wraps back to day 0. Default `7`.
`requireStreak`	boolean		When `true`, missing a claim window resets the player's streak to day 0. When `false`, gaps are forgiven. Default `false`.
`allowReclaim`	boolean		When `true` and `requireStreak` is true, the player can claim day 0 immediately after a streak reset. Default `false`.
`rewards`	array		Ordered reward slots. See below.

`frequency` object

Field	What it controls
`type`	Cadence type. See values below.
`interval`	Number of days per window. Required only for `"every_x_days"`.

`frequency.type` values

Value	Reset behaviour
`"daily"`	Window resets each UTC midnight.
`"weekly"`	Window resets each UTC Monday at 00:00 UTC.
`"monthly"`	Window resets on the 1st of each month at 00:00 UTC.
`"every_x_days"`	Fixed-interval windows of `interval` days aligned to the Unix epoch.
`"oneoff"`	Window never resets. The player can claim once, ever. Good for one-time welcome tracks.

`rewards[]` — RewardSlot shape

Field	Required	What it controls
`day`	✓	Zero-based slot index (0 = first day, 6 = last day of a 7-day cycle). The resolver picks the slot matching `playerStreak % cycleLength`.
`gift`	✓	A `GiftSpec` — either `{ "giftDefId": "..." }` or an inline gift. See GiftSpec (Inline Gifts).

QuestDef and Achievements

Admin table: `questDef`

QuestDef rows define both **quests** and **achievements**. The `category` field distinguishes them.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `quest_boss_hunter`.
`name`	string	✓	Player-facing display name.
`description`	string		Player-facing description of what needs to be done.
`category`	`"quest"` | `"achievement"`		`"quest"` (default) — Quests panel. `"achievement"` — Achievements panel; lifetime one-shot, never resets.
`tier`	number		Achievement prestige tier: `1` = Bronze, `2` = Silver, `3` = Gold. Used for visual theming. Default `1`.
`cadence`	string		Auto-reset schedule. `"oneoff"` (default) = never auto-resets. `"daily"` = resets each UTC midnight. `"weekly"` = resets each UTC Monday. Always `"oneoff"` for achievements.
`enabled`	boolean		Default `true`.
`sortOrder`	number		Display ordering. Lower = shown first.
`repeatable`	boolean		When `true` (and `cadence` is `"oneoff"`), the quest resets after being claimed so the player can complete it again. Default `false`.
`requirements`	`RichSelection`		Gates whether the quest is **visible** to the player at list time.
`objectives`	array	✓	What the player must do to complete the quest/achievement. See below.
`rewardSpec`	`OutcomeEntry[]`		Rewards granted on claim. Same format as mission rewards.
`permanentEffectSpec`	object		**Achievements only.** Permanent `ActiveEffect` granted once on first claim. See below.

`objectives[]` — Objective shape

Field	Required	What it controls
`eventType`		Fast-path filter. Objective only increments when a telemetry event of this type fires. Omit to match any event (rely on `selector` for filtering).
`selector`		`RichSelection` tree evaluated against the combined context (player, entities, inventory, locations, and the live `event` scope).
`target`	✓	Number of qualifying increments required to complete this objective.
`description`		Optional label shown to the player, e.g. `"Complete 3 boss missions"`.

The `event` scope inside an objective selector lets you match specific events:

Field path	Resolves to
`event.type`	The telemetry event type string, e.g. `"mission_complete"`
`event.key`	The event's key value (e.g. a `missionDefId`), or `null`
`event.amount`	Numeric amount carried by the event (default `1`)
`event.data.<path>`	A nested value inside the event's `data` payload

Example objectives:

Goal	Selector sketch
Complete any mission	`event.type eq "mission_complete"`
Complete a specific mission 3 times	AND `event.type eq "mission_complete"` + `event.key eq "mission_boss_alpha"`, target `3`
Win with a Ninja on the team	AND `event.type eq "mission_complete"` + `team.traits.archetype any eq "Ninja"`
Craft at Keeper Level 10+	AND `event.type eq "crafting_completed"` + `player.keeperLevel gte 10`

`permanentEffectSpec` — Achievement Permanent Effect

Grants a lifetime bonus when an achievement is claimed for the first time.

Field	What it controls
`effectDefId`	Which `EffectDef` to activate.
`targetType`	Target of the effect. Falls back to the def's own `targetType` if omitted.
`targetId`	Explicit target ID. Falls back to the claiming player's user ID if omitted.
`value`	Override the def's `value`.
`stats`	Override the def's `stats` distribution.

HomeBannerDef

Admin table: `homeBannerDef`

Home banners are rotating promotional cards on the home screen.

Field	Type	What it controls
`id`	string	Unique identifier.
`name`	string	Internal admin name.
`title`	string	Banner heading shown to players.
`subtitle`	string	Short subheading or countdown label.
`description`	string	Longer body copy (also used as tagline fallback).
`backgroundImage`	string	Asset path or URL for the card background image.
`ctaLabel`	string	Call-to-action button label.
`ctaRoute`	string	Client-side route the CTA and card tap navigate to.
`priority`	number	Display order — higher values appear first.
`startsAt`	datetime	Banner is hidden before this time.
`endsAt`	datetime	Banner is hidden after this time.
`status`	`"active"` | `"inactive"`	`"active"` = shown, `"inactive"` = hidden.
`seasonId`	string	Linked season.
`config`	JSON	Behaviour options (see below).
`display`	JSON	Visual customisation (see below).

`config` — Behaviour options

Key	Type	Default	What it controls
`autoAdvance`	boolean	`true`	Set `false` to disable auto-advance while this banner is in the set.
`autoAdvanceMs`	number	`7000`	Override the auto-advance interval in milliseconds. The carousel uses the highest value across all active banners.
`tagline`	string	—	Short tagline rendered below the subtitle. Overrides `description` as tagline.
`deriveFromActiveSeason`	boolean	`false`	When `true`, `title`, `endsAt`, `startsAt`, and `seasonId` are pulled from the current active Season row. Banner hides if no active Season exists.

`display` — Visual customisation

All keys are optional. Omitting any key falls back to the default purple RPG theme.

Key	Default	What it controls
`titleColor`	`#f0e6ff`	CSS colour for the title.
`subtitleColor`	`#c4b5fd`	CSS colour for the subtitle/countdown.
`taglineColor`	`rgba(255,255,255,0.6)`	CSS colour for the tagline/description.
`accentColor`	`rgba(139,92,246,0.3)`	Card border colour and hover glow.
`cardBackground`	`#0f0a28`	Solid background colour used when no `backgroundImage` is set.
`overlayGradient`	dark purple diagonal gradient	Full CSS gradient rendered over the background image.
`ctaVariant`	`"primary"`	Pre-built CTA button style: `"primary"`, `"gold"`, `"outline"`, `"ghost"`.
`ctaBackground`	—	Custom CSS background for the CTA button. Overrides `ctaVariant`.
`ctaTextColor`	—	Custom text colour for the CTA button. Overrides `ctaVariant`.
`badge`	—	Short chip text in the top-right corner, e.g. `"NEW"`, `"LIMITED TIME"`.
`badgeColor`	`#7c3aed`	Background colour of the badge chip.
`textAlign`	`"left"`	Text alignment: `"left"`, `"center"`, or `"right"`.
`imageHeight`	`170`	Card height in pixels when a background image is present.
`imageObjectPosition`	`"center"`	CSS `object-position` for the background image, e.g. `"top"`, `"50% 20%"`.

EffectDef

Admin table: `effectDef`

An EffectDef is the **reusable template** for any game effect — boosts, multipliers, stat changes, immunity grants, and more. Effects are referenced by ID from consumable items, global powers, achievement rewards, and equipment.

Field	Type	Required	What it controls
`id`	string	✓	Unique snake_case slug, e.g. `xp_boost_flat`.
`name`	string	✓	Human-readable display name.
`description`	string		Admin notes describing the effect's purpose.
`category`	`EffectCategory`	✓	Which game system this effect modifies. See EffectCategory Enum.
`valueMode`	`EffectValueMode`	✓	How the `value` number is applied. See EffectValueMode Enum.
`durationModel`	`EffectDurationModel`	✓	How the effect's lifetime is tracked. See EffectDurationModel Enum.
`stackingRule`	`EffectStackingRule`	✓	What happens when the same effect is applied again. See EffectStackingRule Enum.
`targetType`	`EffectTargetType`	✓	Which game object the effect acts upon. See EffectTargetType Enum.
`value`	number		Primary scalar magnitude. Required for most categories; omit for `stat_boost` (use `stats`) and `trait_grant` (use `trait`).
`stats`	`EffectStatMap`		Per-stat distribution. Used by `stat_boost`. See EffectStatMap.
`trait`	string		Trait archetype name to grant. Required for `trait_grant`.
`durationSec`	number		Duration in real seconds. Required when `durationModel` is `timed`.
`durationTicks`	number		Duration in game ticks. Required when `durationModel` is `tick_based`.
`maxStacks`	number		Max simultaneous stacks. Required for `stack_additive`, `stack_multiplicative`, `stack_capped`.
`maxUses`	number		Max trigger events before the effect expires. Required when `durationModel` is `count_based`.
`cap`	number		Upper bound on accumulated value when `stackingRule` is `stack_capped`.
`resourceId`	string		Scopes the effect to a specific resource (currency or item ID). Used by `production_rate`, `currency_multiplier`, `storage_capacity`, and similar.
`sessionType`	string		Session discriminator for `session` durationModel.
`iconKey`	string		Asset key for the effect icon.

Field	Type	Required	What it controls
`enabled`	boolean		Default `true`.

EffectCategory Enum

Value	Which game system is affected
`stat_boost`	Entity combat stats (ATK, DEF, SPD, FOC, LCK, VIT). Use `stats` field instead of `value`.
`mission_power`	Team's total mission power score.
`success_chance`	Mission success-chance roll.
`reward_multiplier`	Quantity of mission/crafting rewards.
`xp_multiplier`	Entity XP earned from any source.
`keeper_xp_multiplier`	Keeper (player account) XP earned.
`production_rate`	Production rate of a specific resource. `resourceId` required.
`storage_capacity`	Storage cap for a specific resource. `resourceId` required.
`timer_speed`	Global timer speed — applies to all timed activities at once.
`mission_timer_speed`	Timer speed for missions only.
`crafting_timer_speed`	Timer speed for crafting only.
`building_build_timer_speed`	Timer speed for new building construction.
`building_upgrade_timer_speed`	Timer speed for building upgrades.
`injury_recovery_speed`	Timer speed for entity injury recovery.
`dusting_timer_speed`	Timer speed for item dusting jobs.
`mission_timer_speed_boost`	Dedicated mission timer boost (stacks separately from `mission_timer_speed`).
`crafting_timer_speed_boost`	Dedicated crafting timer boost.
`building_build_timer_speed_boost`	Dedicated building build speed boost.
`building_upgrade_timer_speed_boost`	Dedicated building upgrade speed boost.
`dusting_timer_speed_boost`	Dedicated dusting speed boost.
`mission_duration_reduction`	Reduces mission duration by a flat or percentage amount.
`crafting_duration_reduction`	Reduces crafting duration.
`building_build_duration_reduction`	Reduces new building construction time.
`building_upgrade_duration_reduction`	Reduces upgrade construction time.
`dusting_duration_reduction`	Reduces dusting job time.
`injury_recovery_reduction`	Reduces the time or cost of injury recovery.
`trait_grant`	Grants a trait to the target entity. `trait` field required.
`currency_multiplier`	Multiplies currency earned from a specific source. `resourceId` = currency ID.

Value	Which game system is affected
`cost_discount`	Reduces the cost of a specific action.
`slot_unlock`	Unlocks additional slots (e.g. equipment slots). `value` = count to unlock.
`protection`	Provides a protection shield reducing damage or loss.
`reveal`	Reveals hidden information or entities.
`threat_reduction`	Reduces threat level at locations.
`craft_luck`	Adds to the entity's crafting luck (CL) for quality tier rolls.
`worker_efficiency`	Multiplies output of worker assignments at locations.
`cooldown_reduction`	Reduces action cooldowns.
`level_up_bonus`	Awards bonus stats on level-up.

EffectValueMode Enum

Controls how the `value` number is applied to a base value.

Value	Formula	Plain-language
`flat_add`	`base + value`	Adds directly. `+10 ATK`.
`percent_add`	`base × (1 + value/100)`	Adds a percentage. `value = 50` → +50%. Negative values reduce.
`multiplier`	`base × value`	Multiplies. `value = 1.5` → +50%; `value = 2.0` → double.
`flat_set`	`value`	Replaces the base outright, ignoring prior value.
`bps`	`base × (1 + value/10000)`	Basis points. `value = 100` = +1%. For fine-grained adjustments.
`flat_grant`	grants `value` units	Grants units of something rather than modifying a stat.
`random_range`	`random(0, value)`	Upper bound of a uniform random range resolved at activation.

EffectDurationModel Enum

Controls how long an effect lasts.

Value	Required extra fields	How it expires
`instant`	—	Fires once and immediately expires. No persistent record created. Use for one-time grants.
`timed`	`durationSec`	Expires after `durationSec` real seconds.
`tick_based`	`durationTicks`	Expires after `durationTicks` game ticks.
`count_based`	`maxUses`	Expires after being triggered `maxUses` times.
`session`	`sessionType`	Tied to a named session; expires when that session ends.
`permanent`	—	Never expires. Used for achievement rewards and one-time unlocks.
`until_removed`	—	Persists until explicitly removed by admin tool or a game event (e.g. equipment unequip).

EffectStackingRule Enum

Controls what happens when the same effect is applied to the same target a second time.

Value	What happens
`replace`	The new application replaces any existing active effect with the same `effectDefId`. Only one copy active at a time.
`refresh`	The new application resets the timer of the existing effect without changing its value.
`stack_additive`	Each application adds a new stack up to `maxStacks`. Effective value accumulates: `value × currentStacks`.
`stack_multiplicative`	Each application adds a stack up to `maxStacks`. Value compounds multiplicatively across stacks.
`stack_capped`	Stacks accumulate but the total effective value is capped at `cap`.
`no_stack`	Only one copy may be active. A second application attempt is rejected with an error.

EffectTargetType Enum

Controls which game object the effect acts upon.

Value	Description
`entity`	A single entity instance.
`player`	The player account (keeper-level modifiers, currency multipliers).
`location`	A location instance (production rate, storage capacity).
`team`	A mission team as a whole.
`job`	A specific active job.
`global`	All targets of the underlying scope at once. Used by Global Active Powers.

EffectStatMap

Used in the `stats` field on `EffectDef`, `PassiveEffectSpec`, and `EffectStatMap` overrides. Each key is a stat name; the value is the point contribution when the effect is applied.

Key	Stat
`ATK`	Attack
`DEF`	Defence
`SPD`	Speed
`FOC`	Focus
`LCK`	Luck
`VIT`	Vitality
`RND`	**Special.** Each `RND` entry independently picks a random visible stat at award time and applies its full value to that stat. Use a plain number for one draw; use an array for multiple independent draws.

Example — +10 ATK, +5 DEF, one random stat draw of +3:

```
{ "ATK": 10, "DEF": 5, "RND": 3 }
```

EffectDefOverrides

When a consumable item or global power applies an `EffectDef`, you can supply overrides that shadow the def's values for that specific application only. Only these fields can be overridden — structural fields (`category`, `valueMode`, `durationModel`, `stackingRule`, `targetType`) cannot be changed.

Field	What it overrides
`value`	The def's `value`.
`stats`	The def's `stats` distribution.
`durationSec`	`durationSec` (meaningful for `timed` only).
`durationTicks`	`durationTicks` (meaningful for `tick_based` only).
`maxStacks`	The `maxStacks` cap.
`maxUses`	`maxUses` (meaningful for `count_based` only).
`cap`	The stacking value cap.
`resourceId`	The resource scope.

PowerDef

Admin table: `powerDef`

A PowerDef is a **reusable template for a global power event** (a site-wide server-side boost or effect that affects many players at once). Global Active Powers can reference a PowerDef as their default configuration.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `xp_boost_30min`.
`name`	string	✓	Admin-facing display name.
`description`	string		Admin notes.
`powerType`	enum	✓	Mechanical category. See values below.
`effectDefId`	string | null		Required when `powerType = "effect_backed"`. `null` for shorthand types.
`defaultValue`	number		Default effect magnitude.
`defaultValueMode`	string		Default `EffectValueMode`. Default `"percent_add"`.
`defaultDurationMinutes`	number		Default activation duration in minutes. Default `30`.
`defaultDailyLimit`	number		Default maximum times per day this power can fire. Default `1`.
`defaultScope`	string		Default target scope, e.g. `"global"`, `"self"`.
`selection`	JSON		Default `EffectSelection` object applied when the trigger does not supply its own.
`enabled`	boolean		Default `true`.

`powerType` values

Value	`effectDefId`	What it does
`tick_speed_boost`	`null` (required)	Speeds up the global game tick rate.
`effect_backed`	required	Delegates fully to an `EffectDef`. Use when you need fine-grained control over stacking, targeting, or duration.
`keeper_xp_boost`	`null`	Multiplies keeper XP. Shorthand — no `EffectDef` needed.
`entity_xp_boost`	`null`	Multiplies entity XP. Shorthand.
`mission_power_boost`	`null`	Adds to team mission power. Shorthand.
`mission_duration_reduction`	`null`	Reduces mission duration. Shorthand.
`crafting_duration_reduction`	`null`	Reduces crafting duration. Shorthand.
`building_build_duration_reduction`	`null`	Reduces new-build construction time. Shorthand.
`building_upgrade_duration_reduction`	`null`	Reduces upgrade construction time. Shorthand.
`injury_recovery_boost`	`null`	Speeds up injury recovery. Shorthand.

ModalDef and DefaultModalDef

Admin tables: `modalDef` (editable), `defaultModalDef` (update-only; seeded by the system)

ModalDef rows store **server-rendered modal specifications**. Any system that opens a modal by ID — missions, gifts, check-ins, admin triggers — references one of these rows. All string values inside a spec support `${path.to.value}` tokens resolved server-side.

`DefaultModalDef` rows are seeded by the game system (e.g. mission outcome templates). They can be updated but not created or deleted via the admin UI.

Field	Type	Required	What it controls
`id`	string	✓	Stable slug, e.g. `modal_mission_complete`.
`name`	string	✓	Admin-facing display name.
`description`	string		Admin notes.
`spec`	`ModalSpecV1`		The full modal specification. See below.
`enabled`	boolean		Disabled modals return `404` when fetched by the client. Default `true`.

`ModalSpecV1` structure

Field	Required	What it controls
`version`	✓	Always `1`.
`id`	✓	Stable identifier echoed in the response for client-side cache keying.
`kind`	✓	Semantic category, e.g. `"gift_reveal"`, `"mission_outcome"`, `"narrative"`. Informational.
`dismiss`	✓	`{ backdrop, x, escape }` — booleans controlling which interactions close the modal.
`header`	✓	Top-bar content. `{ title, subtitle?, icon?, banner? }`. Supports `${...}` tokens.
`buttons`	✓	Default action buttons shared by all pages. See button fields below.
`pages`	✓	One entry per page. Single-page modals have exactly one entry.
`showPageIndicator`		Show dot-style page progress indicator for multi-page modals. Default `false`.
`telemetry`		Optional tracking metadata.

`ModalButton` fields

Field	Required	What it controls
`id`	✓	Unique within the modal.
`label`	✓	Button text. Supports `${...}` tokens.
`variant`	✓	Visual style: `"primary"`, `"secondary"`, `"ghost"`, `"danger"`.
`disabled`		Renders the button as non-interactive.
`action`	✓	What happens when tapped. See action types below.
`requiresConfirm`		Shows an inline confirmation dialog before executing the action. `{ title, body?, confirmLabel?, cancelLabel? }`.
`skipMeta`		Skip/speed-up metadata. `{ baseCosts, skipType }`.

Modal action types

`type`	What happens
`CLOSE_MODAL`	Dismisses the modal.
`NEXT_PAGE`	Advances to the next page.
`PREV_PAGE`	Returns to the previous page.
`NAVIGATE`	Navigates to a client route. `route` field required.
`ACK_NOTIFICATION`	Marks a notification as read.
`COMPLETE_JOB`	Finalises an in-progress job.
`CLAIM_GIFT`	Claims a gift instance.
`OPEN_INVENTORY`	Opens the inventory screen.
`OPEN_MISSIONS`	Opens the missions screen.
`OPEN_CRAFTING`	Opens the crafting screen.
`REPEAT_MISSION`	Re-dispatches the same mission.
`CRAFT_AGAIN`	Re-runs the same crafting recipe.
`API_CALL`	Fires a registered API action key server-side.
`BUILD_BUILDING`	Starts construction of a building.
`UPGRADE_BUILDING`	Starts a building upgrade.
`SHOP_PURCHASE`	Purchases a shop item.
`DEPOSIT_LOCATION_XP`	Deposits XP charges into a location.
`PRESTIGE`	Triggers player prestige.
`SKIP`	Skips a timed activity using `skipMeta` cost.
`EXECUTE_COMMAND`	Executes an arbitrary game command server-side.

`ModalPageSpec` fields

Each page inherits `header`, `buttons`, and `dismiss` from the root spec and can override them:

Field	What it controls
`body`	Required. Array of `ModalBlock` objects rendered in the page body. For the full block vocabulary, see `render-blocks.md`.
`header`	Partial override of the root header for this page only.
`buttons`	Replaces the root buttons entirely for this page.
`dismiss`	Overrides individual dismiss flags for this page.

For the full list of available body blocks (`richText`, `image`, `timer`, `progress`, `rewardsList`, `entityRoster`, `clickReveal`, etc.) see `render-blocks.md`.

DisplayDef — `style`

Admin table: `displayDef` — the `style` JSON blob

`DisplayDef.style` controls visual overrides for any entity that references the def (Mission Category cards, Mission hero headers, etc.). All sub-objects are optional; omitted keys fall back to hardcoded defaults.

Top-level keys

Key	What it controls
`overlay`	Colour/gradient rendered on top of the background image.
`background`	CSS background-image layout.
`title`	Card heading / hero title font overrides.
`body`	Body / inline text font overrides.
`inlineButtons`	Inline button placement and per-state styling (Mission Category only).

`overlay`

Field	What it controls
`color`	Solid CSS colour, e.g. `#0F0A28`.
`opacity`	Overlay opacity (0–1).
`gradient`	Raw CSS gradient — overrides `color` + `opacity` when set.

`background`

Field	What it controls
`position`	CSS `background-position`.
`size`	CSS `background-size`.
`repeat`	CSS `background-repeat`.
`attachment`	CSS `background-attachment`.

`title` / `body`

Field	What it controls
`fontFamily`	CSS font-family stack.
`fontWeight`	CSS font-weight.
`fontSize`	CSS font-size, e.g. `"22px"`.
`color`	CSS colour.
`letterSpacing`	CSS letter-spacing (`title` only).
`textTransform`	CSS text-transform (`title` only).
`textShadow`	CSS text-shadow (`title` only).
`lineHeight`	CSS line-height.

`inlineButtons`

Controls the inline mission-launch buttons on Mission Category cards.

Field	What it controls
`placement`	`"inline"` (default, beside card content) or `"bottom"` (pushed to card bottom edge).
`normal`	Button style when the player can afford the mission.
`requirementsNotMet`	Button style when the player cannot afford or is gated out.
`disabled`	Button style for explicitly disabled buttons.

Each button state style: `{ background, color, borderColor, opacity }`.

SynergyRule

Admin table: `synergyRule`

A SynergyRule awards **bonuses or penalties based on entity trait combinations and activity context**. When a matching (source, target) entity pair is found on a team, the rule's effects fire.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `syn_ninja_magician`.
`name`	string	✓	Admin-facing display name.
`description`	string		Plain-English explanation of when the rule fires and why.
`enabled`	boolean		Default `true`. Disabled rules are never evaluated.
`sourceSelector`	object		Which entities match the "source" side of the pair. Empty `{}` = any entity.
`targetSelector`	object		Which entities match the "target" side of the pair. Empty `{}` = any entity.
`context`	object		Activity context constraints. Empty `{}` = all activities.
`effects`	array	✓	Effects emitted when the rule fires. See effect types below.
`priority`	number		Evaluation order. Lower = first. Default `100`.
`stackGroup`	string		Group name for cross-rule stacking logic.
`stackMode`	enum		How multiple rule firings are handled. See values below. Default `additive`.
`maxApplications`	number		Max times this rule fires per resolver call. Default `1`.
`startsAt`	datetime		Rule only fires after this time.
`endsAt`	datetime		Rule stops firing after this time.

Selector fields (both `sourceSelector` and `targetSelector`)

All fields are optional. Trait matching is case-insensitive.

Field	What it matches
`entityTypes`	Entities whose `entityType` is in the list.
`speciesIds`	Entities whose `entityDefinitionId` is in the list.
`collections`	Entities from specific NFT collections (`NftCollectionDef.id`).
`traitsAny`	Entities with at least one of these archetype values in any trait slot.
`traitsAll`	Entities that have ALL of these archetype values (each in any slot).
`minTraitCount`	Minimum number of distinct non-null trait values the entity must have.
`entityIds`	Match specific entity instance IDs.
`rarities`	Entities whose rarity is in the list, e.g. `["rare", "legendary"]`.
`regions`	Entities currently stationed in one of these regionIds.
`locationTypes`	Entities currently in a location of one of these types.
`buildingTagsAny`	Entities at a location that has at least one of these building tags.
`missionTagsAny`	Entities with at least one of these mission tags.
`factions`	Entities whose `traits.faction` is in the list.

`context` fields

Field	What it restricts to
`activityTypes`	Specific activity types, e.g. `["mission", "crafting"]`.
`missionTypes`	Specific mission types, e.g. `["boss", "quick"]`.
`missionTagsAny`	Activities tagged with at least one of these mission tags.
`regionIds`	Activities occurring in one of these regions.
`seasonIds`	Activities occurring during one of these seasons.
`requiresTeamSizeMin`	Rule only fires when the team has at least this many entities.
`requiresTeamSizeMax`	Rule only fires when the team has at most this many entities.

Effect types

`type`	Extra fields	What it does
`synergy_score_modifier`	`operation: "flat_add"`, `value`	Adds/subtracts from the team's total synergy score.
`stat_modifier`	`stat` (e.g. `"ATK"`), `operation`, `value`	Modifies an entity stat. Operations: `flat_add`, `percent_add`, `multiplier`.
`duration_modifier`	`activity`, `operation`, `value`	Modifies activity duration (negative = faster). Operations: `flat_seconds`, `percent_add`, `multiplier`.
`resource_yield_modifier`	`resourceId`, `operation`, `value`	Modifies resource output. Operations: `flat_add`, `percent_add`, `multiplier`.
`immunity`	`immuneTo`	Grants immunity to a named debuff or effect type.

`stackMode` values

Value	Behaviour
`additive`	Each matching (source, target) pair triggers the rule independently, up to `maxApplications`.
`unique`	Rule fires exactly once if any pairs match, regardless of pair count.
`best_only`	Within the `stackGroup`, only the rule with the highest positive effect value fires.
`highest_abs_only`	Within the `stackGroup`, only the rule with the highest absolute effect value fires.

SelectionRuleDef

Admin table: `selectionRuleDef`

A SelectionRuleDef is a **named, reusable condition** stored in the database. Reference it from any `RichSelection` tree using `{ "type": "rule_ref", "id": "..." }` to avoid duplicating complex conditions across many definitions.

Field	Type	Required	What it controls
`id`	string	✓	Unique identifier, e.g. `rule_ninja_team`.
`name`	string	✓	Admin-facing display name.
`description`	string		Plain-English explanation of what the condition checks.
`tags`	string[]		Classification tags.
`selection`	`RichSelection`	✓	The condition tree this rule encapsulates.
`enabled`	boolean		Default `true`. A disabled rule always evaluates to `false` when referenced.

Behaviour notes: - Rules are cached in memory at startup. Changes take effect immediately without a server restart. - Circular references (rule A references rule B which references rule A) are detected at resolve time and short-circuit to `false`.

RichSelection — Condition Tree

A `RichSelection` is the **universal condition format** used throughout the game for requirements, objectives, quest selectors, telemetry filters, synergy conditions, and screen visibility. It is a boolean tree of nested groups.

Every `RichSelection` object has two required fields:

Field	Type	What it controls
`op`	`"AND"` | `"OR"` | `"NOT"`	Boolean operator applied to all child rules. `NOT` inverts a single child.
`rules`	array	Child nodes — each is a sub-group, a leaf rule, or a `rule_ref`.

Optional metadata on any node:

Field	What it is
`label`	Admin-facing description of this group.
`description`	Longer explanation.
`failText`	Text shown to the player when this condition is not met.
`passText`	Text shown when it passes.

RichSelectionRule — Leaf Condition

A leaf condition node inside a `RichSelection.rules` array. It reads a value from a scope object and compares it.

Field	Required	What it controls
`scope`	✓	Which context object to read from. See scope values below.
`field`	✓	Dot-path field on the scope object, e.g. `"keeperLevel"`, `"flags.beta_tester"`.
`operator`	✓	Comparison operator. See operator values below.
`value`		Comparison value (number, string, boolean, or array).
`aggregate`		How to reduce multiple values for multi-entity scopes. See aggregate values below.
`where`		A nested `RichSelection` filter applied to narrow the scope before aggregation.
`caseSensitive`		For string comparisons. Default `false`.
`failText`		Text shown when the condition fails.
`passText`		Text shown when it passes.
`id` / `label`		Optional stable identifier / admin label for this node.

`scope` values

Scope	Reads from
`player`	The requesting player's account (e.g. `keeperLevel`, `flags.<key>`).
`entity`	Entity instances owned by the player.
`entity_slot`	Equipment slots on entities.
`team`	The entities currently on the mission team.
`team_member`	A specific team member.
`inventory`	The player's inventory.
`currency_balance`	The player's currency balances.
`item`	A specific item in inventory.
`location`	A location instance.
`currentLocation`	The player's current location.
`building`	A building at a location.
`room`	A room within a building.
`job`	An active job.
`mission`	The current mission.
`recipe`	The current recipe.
`season`	The current active season.
`event`	The telemetry event being processed (quest objectives only).
`context`	Contextual metadata, e.g. `regionId`, `missionType`.
`tempflag`	Ephemeral flags set during the current resolution walk.

`operator` values

Operator	Meaning
`eq`	Equals.
`neq`	Not equals.
`gt`	Greater than.
`gte`	Greater than or equal.
`lt`	Less than.
`lte`	Less than or equal.
`in`	Value is in the provided array.
`not_in`	Value is not in the provided array.
`contains`	String contains substring.
`not_contains`	String does not contain substring.
`exists`	Field is present and not null.
`not_exists`	Field is absent or null.
`between`	Value is between two bounds (provide as array `[min, max]`).
`starts_with`	String starts with the value.
`ends_with`	String ends with the value.
`regex`	String matches regex pattern.

`aggregate` values (for multi-entity scopes)

Aggregate	Meaning
`self`	Default. Reads from the specific scope object directly.
`any`	True if the condition is met by at least one entity.
`all`	True if the condition is met by every entity.
`none`	True if no entity meets the condition.
`count`	Compares the count of matching entities.
`sum`	Sums the field across all entities and compares.
`avg`	Averages the field across all entities and compares.
`min`	Uses the minimum value across entities.
`max`	Uses the maximum value across entities.
`distinct_count`	Counts distinct values.
`ratio`	Ratio of matching entities to total entities.

`rule_ref` node

Reference a `SelectionRuleDef` by ID to embed it inline:

```
{ "type": "rule_ref", "id": "rule_ninja_team", "label": "Optional admin label" }
```

GameDefinition (Screen Compositions & Content Documents)

Admin table: `gameDefinition`

`GameDefinition` rows power two types of server-driven UI: **Screen Compositions** (which sections appear on a page) and **Content Documents** (ordered block lists for specific surfaces like the mission planning panel).

Field	Type	What it controls
`type`	`"screen_composition"` | `"content_document"`	Definition type.
`key`	string	Stable key, e.g. `home_screen` or `mission_planning_document`.
`version`	number	Increment to create a new draft without overwriting the live version.
`isActive`	boolean	Use the publish action — direct mutation risks two active versions.
`startsAt` / `endsAt`	string	ISO datetime availability window. Omit for always-active.
`data`	JSON	Payload — either a `ScreenCompositionDefinition` or a `ContentDocumentDefinition`.

Screen Composition Sections

A `ScreenCompositionDefinition` contains an array of `sections`. Each section places a component on the page.

```
{
  "id": "my-section",
  "component": "HelpTipCard",
  "priority": 10,
  "props": { "title": "Pro Tip", "message": "Check back daily." }
}
```

Section-level fields

Field	What it controls
`id`	Stable identifier for this section entry.
`component`	The registered component key to render. See component list below.
`priority`	Vertical ordering — higher priority = rendered first (top).
`props`	Component-specific configuration object.
`visibility`	Lightweight path/operator/value evaluator for simple level gates and platform targeting.
`richRequirements`	Full `RichSelection` tree for complex requirements (currency, flags, item ownership). Hides the section if conditions are not met.

`visibility` quick format

```
{
  "visibility": {
    "all": [{ "path": "user.keeperLevel", "operator": "greater_than_or_equal", "value": 5 }]
  }
}
```

Supported paths: `user.keeperLevel`, `user.id`, `wallet.isConnected`, `platform`.

Available screen components (component keys)

Component Key	What it renders
`BannerCarousel` / `SeasonBanner`	Swipeable banner carousel sourced from home banners.
`HelpTipCard`	A single dismissible tip card. Props: `title`, `message`.
`MissionCard`	A specific mission card. Props: `missionDefId`.
`RecipeCard`	A specific recipe card. Props: `recipeDefId`.
`TelemetryTracker`	Live event counter display. Props: `telemetryDefId`.
`LocationCard`	A specific location card. Props: `locationDefId`.
(others)	See `render-blocks.md` for the full registry.

Content Document Blocks

A `ContentDocumentDefinition` contains an ordered array of `blocks`. Each block is an object with a `type` field.

Mission-specific blocks

Type	Description
`mission_hero`	Mission banner/title card.
`mission_stats`	Duration, difficulty, AP cost.
`mission_requirements`	Entity/item requirements list.
`mission_synergy`	Team synergy display.
`team_section`	Entity slot grid (editable or locked by context).
`reward_list`	Expected / actual rewards.
`entity_roster`	Read-only entity summary.
`mission_timer`	Countdown to completion.
`outcome_banner`	Success/failure hero banner.
`narrative_text`	Optional flavour text.

Generic blocks

Type	Fields	Description
`rich_text`	`content?`	Freeform rich text content.
`divider`	—	Visual separator.
`image`	`imageUrl?`, `alt?`	Inline image.

`SurfaceContextOverrideRule`

Declared in the screen/modal config, not in the document itself. Overrides a slice of the document context when a live event fires.

```
{
  "on": "job.completed",
  "jobId": "optional-job-id",
  "apply": { "missionMode": "outcome", "teamMode": "locked", "rewardMode": "actual" }
}
```

Supported triggers: `job.completed`, `job.failed`, `job.ready`.

TagDef

Admin table: `tagDef`

TagDef rows define content classification tags used to group and filter definitions across the admin panel.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `tag_seasonal`.
`name`	string	✓	Display label.
`description`	string		Admin notes about when to apply this tag.
`enabled`	boolean		Default `true`.

GameConstant

Admin table: `gameConstant`

GameConstant rows are **server-side tuning knobs** — named numbers or strings that the game engine reads at runtime. Change a constant here to tune behaviour without a code deployment.

Field	Type	Required	What it controls
`id`	string	✓	Unique slug, e.g. `SPD_TIMER_SCALE`.
`value`	string | number	✓	The constant value read by the engine.
`description`	string		Plain-English explanation of what this constant affects and safe value ranges.
`enabled`	boolean		Default `true`. A disabled constant falls back to the code default.

Key constants (examples)

Constant ID	What it does
`SPD_TIMER_SCALE`	How much each point of SPD reduces a mission timer. Higher = faster missions for high-SPD teams.

> Always fill in `description` when creating a new constant so that future editors know what the knob does and what values are safe.

This guide covers all game definition types as of the current codebase. Cross-references link to the relevant sections. For UI block vocabulary (modal body blocks, screen composition blocks), see `render-blocks.md`.

Worked Examples

The following end-to-end examples show how multiple definition types fit together for common content scenarios. Each example lists every definition you need to create or edit, explains why each field is set the way it is, and calls out the cross-references that wire things together.

Example 1 — Multi-Phase Expedition Mission with Tiered Rewards

Scenario: A three-phase expedition called "Ruins of Vaelthorn" where teams push deeper into a dungeon. Each phase is progressively harder. Rewards scale with the team's Mission Power, using the Gaussian tier system. Players can choose to press on or retreat after each phase.

What you will create

#	Definition type	ID
1	`MissionDef`	`expedition_vaelthorn`
2	`ModalDef` (lead-in)	`modal_vaelthorn_intro`
3	`ModalDef` (phase decision)	`modal_vaelthorn_phase_decision`
4	`BundleDef` (consolation loot)	`bundle_vaelthorn_consolation`

Step 1 — Create the consolation BundleDef

Any team that is force-abandoned after too many failures gets this small consolation prize. Defining it as a reusable `BundleDef` lets you reference it from the mission's `failure` block without duplicating the entry list.

Admin table: `bundleDef`

```
{
  "id": "bundle_vaelthorn_consolation",
  "name": "Vaelthorn Consolation Loot",
  "bundle": [
    { "type": "currency", "key": "soft_arcanium", "min": 50, "max": 100 },
    { "type": "entity_xp", "min": 200, "max": 400 },
    { "type": "keeper_xp", "min": 100, "max": 100 }
  ]
}
```

Why: `soft_arcanium` is your SOFT currency. `entity_xp` is split across all entities on the team; `keeper_xp` goes to the player account. Both are included because even failed runs should feel rewarding to some extent.

Step 2 — Create the lead-in ModalDef

This modal opens once per session when the player taps the mission card, before they select a team. It sets the narrative scene.

Admin table: `modalDef`

```
{
  "id": "modal_vaelthorn_intro",
  "name": "Vaelthorn — Intro Narrative",
  "spec": {
    "version": 1,
    "id": "modal_vaelthorn_intro",
    "kind": "narrative",
    "dismiss": { "backdrop": true, "x": true, "escape": true },
    "header": {
      "title": "Ruins of Vaelthorn",
      "subtitle": "A darkness stirs beneath the stone."
    },
    "buttons": [
      { "id": "btn_close", "label": "Understood", "variant": "primary", "action": { "type": "CLOSE_MODAL" } }
    ],
    "pages": [
      {
        "body": [
          {
            "type": "richText",
            "doc": [
              { "t": "p", "c": [{ "t": "text", "v": "Ancient catacombs stretch for miles beneath Vaelthorn Kee
            ]
          },
          {
            "type": "richText",
            "doc": [
              { "t": "p", "c": [{ "t": "strong", "c": [{ "t": "text", "v": "This is a multi-phase expedition."
            ]
          }
        ]
      }
    ]
  }
}
```


Why: `kind: "narrative"` is informational — it has no mechanical effect, but it helps admin editors identify modal purpose at a glance. `dismiss.backdrop: true` lets casual players skip past it immediately.

Step 3 — Create the phase-decision ModalDef

This modal opens at each `phase_ready` decision point — after a phase completes successfully and `continueRules.onSuccess` is `"wait"`. The player sees their partial rewards so far and chooses to continue or withdraw.

Admin table: `modalDef`

```
{
  "id": "modal_vaelthorn_phase_decision",
  "name": "Vaelthorn — Phase Decision",
  "spec": {
    "version": 1,
    "id": "modal_vaelthorn_phase_decision",
    "kind": "mission_phase_decision",
    "dismiss": { "backdrop": false, "x": false, "escape": false },
    "header": {
      "title": "Phase Complete",
      "subtitle": "Press deeper, or withdraw?"
    },
    "buttons": [
      {
        "id": "btn_continue",
        "label": "Press Deeper",
        "variant": "primary",
        "action": { "type": "EXECUTE_COMMAND", "command": "ContinueMission", "payload": { "jobId": "${job.id}"
      },
      {
        "id": "btn_abandon",
        "label": "Withdraw",
        "variant": "secondary",
        "action": {
          "type": "EXECUTE_COMMAND",
          "command": "AbandonMission",
          "payload": { "jobId": "${job.id}" }
        },
        "requiresConfirm": {
          "title": "Withdraw from Vaelthorn?",
          "body": [{ "t": "p", "c": [{ "t": "text", "v": "You will collect rewards earned so far, but cannot r
          "confirmLabel": "Yes, Withdraw",
          "cancelLabel": "Stay"
        }
      }
    ],
    "pages": [
      {
        "body": [
          {
            "type": "richText",
            "doc": [{ "t": "p", "c": [{ "t": "text", "v": "Your team has cleared this phase. The next is more
          }
        ]
      }
```

```
    ]
  }
}
```

Why: `dismiss` is entirely locked (`false` on all three flags) because we must force the player to make an explicit choice — an accidental close would leave the job stuck in `phase_ready`. The "Withdraw" button uses `requiresConfirm` as an additional safety net against fat-finger abandons.

Step 4 — Create the MissionDef

Admin table: `missionDef`

```
{
  "id": "expedition_vaelthorn",
  "name": "Ruins of Vaelthorn",
  "description": "A three-phase expedition into the cursed catacombs beneath Vaelthorn Keep. Deeper phases rew
  "missionType": "expedition",
  "regionId": "region_darklands",
  "difficulty": 80,
  "minSuccessChance": 10,
  "duration": 3600,
  "teamMin": 2,
  "teamMax": 4,
  "mode": "multi_phase",
  "rewardsMode": "per_phase",
  "maxFailures": 2,
  "leadInModalDefId": "modal_vaelthorn_intro",
  "phaseDecisionModalDefId": "modal_vaelthorn_phase_decision",
  "failure": {
    "rewards": [
      { "type": "bundle_ref", "bundleDefId": "bundle_vaelthorn_consolation" }
    ]
  },
  "phases": [
    {
      "name": "Outer Ward",
      "description": "Break through the crumbling outer fortifications.",
      "duration": 1800,
      "difficulty": 50,
      "injuryRate": 0.10,
      "injuryPenalty": 0.9,
      "count": 1,
      "tiers": {
        "sigma": 25,
        "common":   { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 80,  "max": 120 }, { "t
        "uncommon": { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 120, "max": 180 }, { "t
        "rare":     { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 200, "max": 280 }, { "t
        "epic":     { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 300, "max": 400 }, { "t
        "failure":  { "rewards": [{ "type": "bundle_ref", "bundleDefId": "bundle_vaelthorn_consolation" }] }
      },
      "continueRules": {
        "onSuccess": "wait",
        "onFailure": "next_phase"
      }
    },
    {
      "name": "Flooded Halls",
```

```
      "description": "Wade through the submerged lower passages. Injuries are more likely here.",
      "duration": 2700,
      "difficulty": 80,
      "injuryRate": 0.25,
      "injuryPenalty": 0.75,
      "count": 1,
      "tiers": {
        "sigma": 22,
        "uncommon": { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 150, "max": 250 }, { "t
        "rare":     { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 280, "max": 380 }, { "t
        "epic":     { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 400, "max": 550 }, { "t
        "mythic":   { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 600, "max": 800 }, { "t
        "failure":  { "rewards": [{ "type": "bundle_ref", "bundleDefId": "bundle_vaelthorn_consolation" }] }
      },
      "continueRules": {
        "onSuccess": "wait",
        "onFailure": "next_phase"
      }
    },
    {
      "name": "Inner Sanctum",
      "description": "Face the corruption at its source. Only the strongest teams survive.",
      "duration": 3600,
      "difficulty": 130,
      "injuryRate": 0.40,
      "injuryPenalty": 0.60,
      "count": 1,
      "tiers": {
        "sigma": 20,
        "rare":      { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 500,  "max": 700  }, {
        "epic":      { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 800,  "max": 1000 }, {
        "mythic":    { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 1200, "max": 1600 }, {
        "legendary": { "rewards": [{ "type": "currency", "key": "soft_arcanium", "min": 2000, "max": 2500 }, {
        "failure":   { "rewards": [{ "type": "bundle_ref", "bundleDefId": "bundle_vaelthorn_consolation" }] }
      },
      "continueRules": {
        "onSuccess": "complete",
        "onFailure": "end_job"
      }
    }
  ],
  "tags": ["expedition", "dungeon", "darklands"],
  "sortOrder": 10,
  "enabled": true
}
```

Key decisions explained:

Decision	Why
`rewardsMode: "per_phase"`	Each phase rolls its own `tiers`, so players who withdraw early still collect loot. A `"final_bundle"` run would give nothing to players who retreat.
`maxFailures: 2`	Two phase failures force-abandon the expedition. This prevents teams from grinding indefinitely through cheap failures to reach phase 3.
`continueRules.onSuccess: "wait"` on phases 1 & 2	Opens the phase-decision modal so the player actively chooses to continue. Without `"wait"`, the game auto-advances immediately.
`continueRules.onFailure: "next_phase"` on phases 1 & 2	A phase failure still advances — the team limps forward. The `maxFailures` cap is what eventually ends the run.
`continueRules.onSuccess: "complete"` and `onFailure: "end_job"` on phase 3	Final phase: success completes the expedition; failure ends it immediately with no final bundle (consolation is handled inside the `failure` tier).
`injuryRate` / `injuryPenalty` escalating per phase	Makes deeper phases genuinely risky. A high-VIT team will absorb more punishment; a low-VIT team feels the snowballing penalty.
`sigma` tightening (25 → 22 → 20)	Inner phases are more decisive — the team's actual power level is a stronger predictor of outcome the deeper they go.

Example 2 — Consumable Item that Applies an XP Boost

Scenario: A consumable called "Scholar's Elixir" that, when used on an entity, grants that entity a 50 % XP multiplier for the next 30 minutes.

What you will create

#	Definition type	ID
1	`EffectDef`	`effect_entity_xp_boost_50pct`
2	`ItemDefinition` (consumable)	`consumable_scholars_elixir`

Step 1 — Create the EffectDef

The EffectDef is the mechanical blueprint for the boost. The item will reference it by ID.

Admin table: `effectDef`

```
{
  "id": "effect_entity_xp_boost_50pct",
  "name": "Entity XP Boost +50%",
  "description": "Multiplies XP earned by the target entity by 1.5x for a fixed duration. Applied by Scholar's
  "category": "xp_multiplier",
  "valueMode": "percent_add",
  "durationModel": "timed",
  "stackingRule": "refresh",
  "targetType": "entity",
  "value": 50,
  "durationSec": 1800,
  "iconKey": "effect_xp_star",
  "enabled": true
}
```

Field-by-field reasoning:

Field	Value	Why
`category`	`xp_multiplier`	This affects entity XP only, not keeper XP or any other system.
`valueMode`	`percent_add`	`percent_add` with `value: 50` means `base × (1 + 50/100)` = ×1.5. Using `multiplier` with `value: 1.5` would also work but `percent_add` is more intuitive for content creators to reason about ("+50%").
`durationModel`	`timed`	The boost is time-limited in real seconds. `durationSec: 1800` = 30 minutes.
`stackingRule`	`refresh`	If the player uses a second elixir while one is active, the timer resets to 30 minutes rather than adding another stack. This prevents the boost from becoming permanent through repeated use.
`targetType`	`entity`	The boost applies to the specific entity the player uses the item on, not to all entities or the player account.

Step 2 — Create the ItemDefinition

Admin table: `itemDefinition`

```
{
  "id": "consumable_scholars_elixir",
  "name": "Scholar's Elixir",
  "description": "A shimmering draught said to sharpen the mind and accelerate growth. Grants the imbiber +50%
  "type": "CONSUMABLE",
  "rarity": "uncommon",
  "maxStack": 99,
  "iconKey": "item_elixir_blue",
  "consumableData": {
    "effects": [
      {
        "effectDefId": "effect_entity_xp_boost_50pct",
        "targetType": "entity"
      }
    ]
  },
  "dustingData": {
    "tiers": {
      "common": {
        "rewards": [
          { "type": "currency", "key": "soft_arcanium", "min": 10, "max": 20 }
        ]
      }
    }
  },
  "enabled": true
}
```

Key decisions explained:

Decision	Why
`type: "CONSUMABLE"`	Required for the `consumableData` block to be read by the engine.
`effects[].targetType: "entity"`	Tells the UI to prompt the player to pick a specific entity before using the item. The value must match the `targetType` on the referenced `EffectDef`.
No `effectOverrides` on the entry	The item uses the EffectDef's default values (50%, 30 min). If you wanted a premium version with 100% for 1 hour, you would add `"effectOverrides": { "value": 100, "durationSec": 3600 }` to the same entry instead of creating a second EffectDef.
`maxStack: 99`	Consumables typically stack. Leaving this at `null` (unlimited) is also fine; `99` is a common cap that prevents inventory clutter.
`dustingData`	Optional but recommended — gives players a recourse if they have too many. Yields a small currency return on breakdown.

Variant: extending to equipment passive

If you later want a piece of gear that permanently applies a weaker version of the same bonus while equipped, add a `passiveEffects` block to the gear item. Unlike consumables, `passiveEffects` entries are **inline effect objects** — they do not reference an `EffectDef` row and do not create `ActiveEffect` records. They are resolved on the fly while the item is equipped.

```
"passiveEffects": [
  { "category": "xp_multiplier", "valueMode": "percent_add", "value": 20 }
]
```

> **Note:** Passive equipment effects are always `until_removed` — they are active as long as the item is equipped and removed the moment it is unequipped. There is no `durationModel` or stacking configuration in `passiveEffects` entries.

Example 3 — 7-Day Check-In Track with Inline Gifts

Scenario: A rolling daily check-in track called "Arcane Attendance". Players earn escalating rewards over 7 consecutive login days, with a special bonus pack on the 7th claim. Missing any day resets progress back to day 0 (slot 0). After the full 7-day cycle is claimed the streak wraps and restarts from day 1.

What you will create

#	Definition type	ID
1	`GiftDef` (day-7 grand prize)	`gift_arcane_attendance_grand`
2	`CheckInDef`	`checkin_arcane_attendance`

Days 1–6 use inline gifts (no database row needed). Day 7 uses a `GiftDef` row so the same grand prize can be referenced from future events without duplication.

Step 1 — Create the day-7 GiftDef

Admin table: `giftDef`

```
{
  "id": "gift_arcane_attendance_grand",
  "name": "Arcane Attendance — Grand Prize",
  "presentation": {
    "title": "Full Attendance Bonus!",
    "subtitle": "You checked in every day this week.",
    "iconKey": "gift_chest_gold",
    "accentColor": "#FFD700"
  },
  "claimRules": {
    "expiresInSeconds": 172800
  },
  "rewardSpec": [
    { "type": "currency", "key": "soft_arcanium", "min": 1000, "max": 1000 },
    { "type": "item",     "key": "consumable_scholars_elixir", "min": 3, "max": 3 },
    { "type": "keeper_xp", "min": 500, "max": 500 },
    {
      "bundle": [
        { "type": "item", "key": "mat_vaelthorn_essence", "min": 1, "max": 1, "odds": 20 },
        { "type": "nothing", "odds": 80 }
      ],
      "pickMin": 1,
      "pickMax": 1
    }
  ],
  "instant": true,
  "enabled": true
}
```

Key decisions explained:

Decision	Why
`claimRules.expiresInSeconds: 172800`	The grand prize expires 48 hours after it is issued. This prevents players from stockpiling unclaimed gifts across multiple weeks.
`instant: true`	Rewards are applied immediately on claim rather than becoming a GIFT_PACKAGE item in inventory. Keeps the flow snappy for a daily reward.
Inline pick-mode bundle for `mat_vaelthorn_essence`	A 20% chance of a rare material adds lottery excitement to the day-7 reward without guaranteeing it. Using an inline `bundle` array with `pickMin/pickMax` keeps it self-contained — no need for a separate `BundleDef` for a one-off use.
`consumable_scholars_elixir` in the rewards	Referencing the item from Example 2 directly — a real-world cross-example dependency that mirrors how the content library grows over time.

Step 2 — Create the CheckInDef

Admin table: `checkInDef`

```
{
  "id": "checkin_arcane_attendance",
  "name": "Arcane Attendance",
  "description": "7-day weekly streak track. Resets each Monday. Consecutive daily claims unlock escalating re
  "enabled": true,
  "frequency": { "type": "daily" },
  "cycleLength": 7,
  "requireStreak": true,
  "allowReclaim": false,
  "rewards": [
    {
      "day": 0,
      "gift": {
        "name": "Day 1 — Arcanium Stipend",
        "presentation": { "title": "Day 1 Bonus", "iconKey": "gift_coin" },
        "rewardSpec": [
          { "type": "currency", "key": "soft_arcanium", "min": 100, "max": 100 }
        ],
        "instant": true
      }
    },
    {
      "day": 1,
      "gift": {
        "name": "Day 2 — Keeper XP",
        "presentation": { "title": "Day 2 Bonus", "iconKey": "gift_scroll" },
        "rewardSpec": [
          { "type": "currency", "key": "soft_arcanium", "min": 150, "max": 150 },
          { "type": "keeper_xp", "min": 100, "max": 100 }
        ],
        "instant": true
      }
    },
    {
      "day": 2,
      "gift": {
        "name": "Day 3 — Entity XP Surge",
        "presentation": { "title": "Day 3 Bonus", "iconKey": "gift_orb" },
        "rewardSpec": [
          { "type": "currency", "key": "soft_arcanium", "min": 200, "max": 200 },
          { "type": "entity_xp", "min": 300, "max": 500 }
        ],
        "instant": true
      }
    },
```

```
    {
      "day": 3,
      "gift": {
        "name": "Day 4 — Material Cache",
        "presentation": { "title": "Day 4 Bonus", "iconKey": "gift_sack" },
        "rewardSpec": [
          { "type": "currency", "key": "soft_arcanium", "min": 250, "max": 250 },
          { "type": "item", "key": "mat_dungeon_stone", "min": 2, "max": 4 }
        ],
        "instant": true
      }
    },
    {
      "day": 4,
      "gift": {
        "name": "Day 5 — Keeper & Entity Boost",
        "presentation": { "title": "Day 5 Bonus", "iconKey": "gift_flask" },
        "rewardSpec": [
          { "type": "currency", "key": "soft_arcanium", "min": 300, "max": 300 },
          { "type": "keeper_xp", "min": 200, "max": 200 },
          { "type": "entity_xp", "min": 500, "max": 700 }
        ],
        "instant": true
      }
    },
    {
      "day": 5,
      "gift": {
        "name": "Day 6 — Scholar's Elixir",
        "presentation": { "title": "Day 6 Bonus", "iconKey": "gift_elixir" },
        "rewardSpec": [
          { "type": "currency", "key": "soft_arcanium", "min": 350, "max": 350 },
          { "type": "item", "key": "consumable_scholars_elixir", "min": 1, "max": 1 }
        ],
        "instant": true
      }
    },
    {
      "day": 6,
      "gift": { "giftDefId": "gift_arcane_attendance_grand" }
    }
  ]
}
```

Key decisions explained:

Decision	Why
`frequency.type: "daily"`	Each individual claim window resets at UTC midnight. The player must log in and claim on each of the 7 calendar days.
`cycleLength: 7`	After 7 claims the streak wraps — day 8 starts back at day 0 (Day 1 reward). This is the loop for an ongoing weekly track.
`requireStreak: true`	Missing a day resets the player to slot 0. This is the key mechanic that makes the track a "streak" rather than a free-roam collection.
`allowReclaim: false`	After a streak break the player cannot double-claim day 0 on the same day they reset. They must wait for the next window. Set to `true` if you want to be more forgiving.
Days 0–5 use inline gifts	These gifts are unique to this track and will not be reused elsewhere, so there is no benefit to creating `GiftDef` rows for them. Inline keeps everything in one place.
Day 6 references `gift_arcane_attendance_grand` by ID	The grand prize is reused in seasonal events and future tracks, so it lives as a `GiftDef` row. The check-in slot just holds `{ "giftDefId": "..." }`.
`day` indices are 0-based	`day: 0` = first claim, `day: 6` = seventh claim. Make sure the number of `rewards` entries equals `cycleLength`. The resolver uses `playerStreak % cycleLength` to pick the slot.

How the three examples connect

The examples above are deliberately cross-referenced to mirror real production content authoring:

- **Example 3** (the check-in track) rewards `consumable_scholars_elixir` on day 6 — the item created in **Example 2**.
- **Example 3**'s day-7 `GiftDef` includes a chance at `mat_vaelthorn_essence` — the rare material dropped in **Example 1**'s inner-sanctum phase.
- **Example 1**'s consolation bundle references `soft_arcanium` — the same currency used throughout both other examples.

This is the normal shape of a game's content graph: items flow from hard content (expeditions) through player progression (level-ups, check-ins) into craft inputs (recipes). Build new content by first identifying where it fits in this graph, then creating definitions from the leaves inward (EffectDefs and BundleDefs first, then Items, then Missions or CheckInDefs that reference them).